

Welcome back! Hybrid Search (Module 4) mein aapne dekha ki BM25 kitna powerful hai exact keyword matching ke liye. Par industry-level engineer banne ke liye sirf BM25 ka function call karna kaafi nahi hai; aapko uske andar ka math aur mechanics samajhna hoga. Aaj hum deep dive karenge **Module 5: BM25 Retrieval** mein. Let's decode the magic of sparse retrieval!

Module 5: BM25 Retrieval

1. Why this concept exists

Information Retrieval (IR) ki duniya mein humesha se ek challenge raha hai: "Kaunsa document user ki query se sabse zyada match karta hai?"

Pehle hum TF-IDF (Term Frequency-Inverse Document Frequency) use karte the. Lekin usme kuch flaws the (jaise agar ek lamba document hai, toh usme word frequency naturally zyada hogi). **BM25 (Best Matching 25)** TF-IDF ka ek highly optimized aur upgraded version hai. Ye exist karta hai taaki hum text-matching ko smart aur fair bana sakein, bina AI/Embeddings use kiye.

2. Problem with traditional RAG (Dense Only)

Basic RAG (Vector Embeddings) "meaning" match karta hai.

Problem: Agar user search kare "Invoice #INV-2023-998", toh vector model confuse ho jayega kyunki numbers aur exact IDs ka koi specific "semantic meaning" nahi hota. Embeddings out-of-vocabulary (OOV) words, names, aur IDs par fail ho jati hain. Yahan BM25 savior banta hai kyunki wo exact character/word tokens dhoondhta hai.

3. Real-world example

Aap ek Medical Record system bana rahe hain.

Doctor search karta hai: *"Patient prescribed with 50mg Metoprolol Tartrate"*

- **Vector Search:** Shayad kisi aur heart medicine (jaise Atenolol) ka document le aaye kyunki dono ka medical meaning similar hai.
- **BM25 Search:** Ye clearly "50mg", "Metoprolol", aur "Tartrate" words ko document mein count karega aur exact patient ka record nikal kar layega.

4. Visual explanation using diagrams made with text

Plaintext

Query: "Apple iPhone 15"

Document 1 (Short): "I bought an Apple iPhone 15 today."

Document 2 (Long) : "Apple makes fruits. I like the apple. iPhone is good. 15 is a number."

How BM25 Thinks

1. TF (Term Frequency) -> Dono docs mein words kitni baar aaye.

2. IDF (Inverse Doc Freq) -> "Apple" common hai, but "iPhone 15" rare hai. Rare words get HIGHER scores.

3. Length Normalization -> Doc 1 chota hai aur usme relevant words hain, so it is dense and gets a BOOST. Doc 2 lamba hai (diluted) gets penalized.

BM25 Score Engine

Doc 1

Doc 1 Score Doc 2 Score

(9.8) (3.2) => Doc 1 Wins

5. Architecture flow

1. **Tokenization:** Documents aur Query ko chote words (tokens) mein toda jata hai (e.g., lowercase, removing punctuation).
2. **TF Calculation:** System check karta hai ki query ke words ek specific document mein kitni baar aaye.
3. **IDF Calculation:** System check karta hai ki wo words poore corpus (saare documents) mein kitne common/rare hain.
4. **Saturation & Normalization (\$k_1\$ & \$b\$):** System lamba/chota hone ke basis par document ke score ko adjust karta hai.
5. **Final Scoring:** Har document ko ek numerical score milta hai aur list sort (rank) ho jati hai.

6. Deep Technical Explanation (TF, IDF, & BM25 Formula)

Isko dhyan se samajhiye, ye senior level ka concept hai:

- **TF (Term Frequency):** Ek word document mein kitni baar aaya. TF-IDF mein, agar word 100 baar aaya, toh score 100x ho jata tha. BM25 isme ek "cap" (saturation) lagata hai. Agar word 5 baar aaya ya 50 baar aaya, ek limit ke baad score badhna band ho jata hai. Is parameter ko k_1 (usually 1.2 to 2.0) kehte hain.
- **IDF (Inverse Document Frequency):** Ek word poore dataset mein kitna rare hai. "The", "Is", "And" har document mein hote hain, so inka IDF score ~0 hota hai. "Metoprolol" rare hai, iska IDF high hoga.
- **Document Length Normalization (\$b\$):** Chote docs naturally kam words rakhte hain. Agar lamba doc hai, toh hum expect karte hain ki words zyada baar aayenge. b parameter (usually 0.75) lambe documents ko penalize karta hai taaki chote aur direct answers wale documents ko fair chance mile.

The BM25 Formula:

Agar humari query Q hai jisme keywords $q_1, q_2, \dots, q_n$ hain, toh ek Document D ka

BM25 score aise nikalta hai:

$$\text{Score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{\text{TF}(q_i, D) \cdot (k_1 + 1)}{\text{TF}(q_i, D) + k_1} \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})$$

(Jahan $|D|$ document length hai aur avgdl average document length hai).

7. Step-by-step implementation & 8. Complete Python / LangChain code

LangChain ne BM25 implementation ko bahut aasan bana diya hai using the rank_bm25 library.

Prerequisite: pip install rank_bm25 langchain-community

Python

```
import re # For better tokenization (optional, but good for some languages)
from langchain_community.retrievers import BM25Retriever
from langchain_core.documents import Document
```

1. Create a dummy dataset (Corpus)

```
texts = [
    "The quick brown fox jumps over the lazy dog.",
    "Apple releases the new iPhone 15 Pro Max with 256GB storage.",
    "Error code ERR-9021 indicates a database connection failure.",
    "A database is an organized collection of data.",
    "The new Apple MacBook Pro is great for developers."
]
```

Convert texts to LangChain Document objects

```
docs = [Document(page_content=t) for t in texts]
```

2. Initialize BM25 Retriever

Ye automatically TF, IDF, k1, aur b calculate karke index bana lega memory mein

```
retriever = BM25Retriever.from_documents(docs)
```

Aap top K results customize kar sakte hain

```
retriever.k = 2
```

3. Test Query 1: Exact ID Search (Perfect for BM25)

```
query1 = "ERR-9021"
```

```
print(f"Query: '{query1}'")
```

```

results1 = retriever.invoke(query1)
for doc in enumerate(results1):
    print(f"Rank {i+1}: {doc.page_content}")

```

```

print("-" 40)

```

```

# 4. Test Query 2: Keyword search with rare term (iPhone)

```

```

query2 = "iPhone 15 storage"
print(f"Query: '{query2}'")
results2 = retriever.invoke(query2)
for doc in enumerate(results2):
    print(f"Rank {i+1}: {doc.page_content}")

```

9. Production use case

Enterprise Search & E-Commerce: Amazon ya Flipkart ki search mostly BM25 (ya uske variants like Elasticsearch) par chalti hai. Agar aap search bar mein "Nike Shoes size 10" type karein, toh unhe 'meaning' nahi, exact 'Nike' brand aur '10' size chahiye. Aise exact keyword matching mein BM25 embeddings ko out-perform karta hai.

10. Interview questions

- Q: BM25 mein \$k_1\$ aur \$b\$ parameters ka kya role hai?**
 A: \$k_1\$ term frequency saturation ko control karta hai (ek limit ke baad word repetition ka benefit kam kar deta hai). \$b\$ document length normalization control karta hai (lambe documents ko fair scoring dene ke liye).
- Q: TF-IDF aur BM25 mein main difference kya hai?**
 A: TF-IDF mein Term Frequency linearly scale karti hai (word 100 baar aaya toh score 100x). BM25 asymptotic saturation use karta hai (word 5-6 baar aane ke baad score plateau ho jata hai taaki keyword stuffing kaam na kare).
- Q: Kya BM25 mein Semantic similarities catch hoti hain? Jaise "Happy" aur "Joyful"?**
 A: Nahi. BM25 purely exact string match par kaam karta hai. Agar document mein "Happy" nahi hai, toh score 0 hoga, bhale hi "Joyful" 100 baar likha ho.

11. Advantages

- No Embedding Model Required:** Token usage nahi hoti, GPU nahi chahiye. Computationally bohot sasta hai.
- Exact Matches:** IDs, phone numbers, error codes dhoondhne ke liye perfect hai.
- Explainable:** Hum easily mathematically bata sakte hain ki kisi document ko high rank kyun mili.

12. Limitations

- **Vocabulary Mismatch:** Synonyms nahi samajhta (e.g., "Car" dhoondhne par "Automobile" wala document nahi milega).
- **Context-Blind:** Words ka context nahi samajhta (e.g., "Apple" fruit hai ya company).
- **Memory Intensive for Local Python:** rank_bm25 saara data RAM mein load karta hai. Billions of docs ke liye ye crash ho jayega.

13. Best practices

- **Text Preprocessing:** BM25 retriever mein data daalne se pehle lowercasing, stemming aur stop-word removal (the, is, at) zaroor lagayein. Isse index chota aur fast banta hai.
- **Don't use rank_bm25 in large-scale production:** LangChain ka BM25Retriever prototyping ke liye hai. Production mein **Elasticsearch**, **OpenSearch**, ya **Milvus** use karein jo efficiently BM25 run karte hain disk-based indexing ke sath.
- **Always Pair with Dense Search:** Module 4 yaad hai na? BM25 ko akele mat use karo. Isko Dense (Vector) search ke sath Hybrid search mein use karo for 100% accuracy.

Assignments & Practical Exercises

- **Exercise 1:** Apne Python code mein 2 same lambai ke documents banao. Ek mein word "Machine" 2 baar likho aur doosre mein 20 baar. Dono ka BM25 score observe karo (Score saturation check karo).
- **Assignment:** Elasticsearch ka architecture padho. Check karo ki wo internally text fields par BM25 kaise apply karte hain (Hint: Inverted Index).

Common Mistakes Beginners Make

- *Passing Embeddings to BM25:* Beginner galti se LangChain mein BM25Retriever mein OpenAI embeddings pass karne lagte hain. Yaad rakhein, BM25 text-based math hai, isko vectors ya embeddings nahi chahiye!
- *Ignoring Chunking:* Log pura 50-page ka PDF ek document banakar BM25 mein daal dete hain. Chunking yahan bhi zaroori hai, warna "b" parameter lambe document ki rank gira dega.

Ye tha BM25 ka deep dive. Ab aapko Hybrid Search ka backend completely samajh aagaya hai!

Ready for Module 6? Let's dive deep into Dense Retrieval (FAISS, Chroma, Semantic Search). Say "Yes" to continue!